Universidade Cruzeiro do Sul
Ciência da Computação

Aula 10 - Paradigma Imperativo – Parte 1

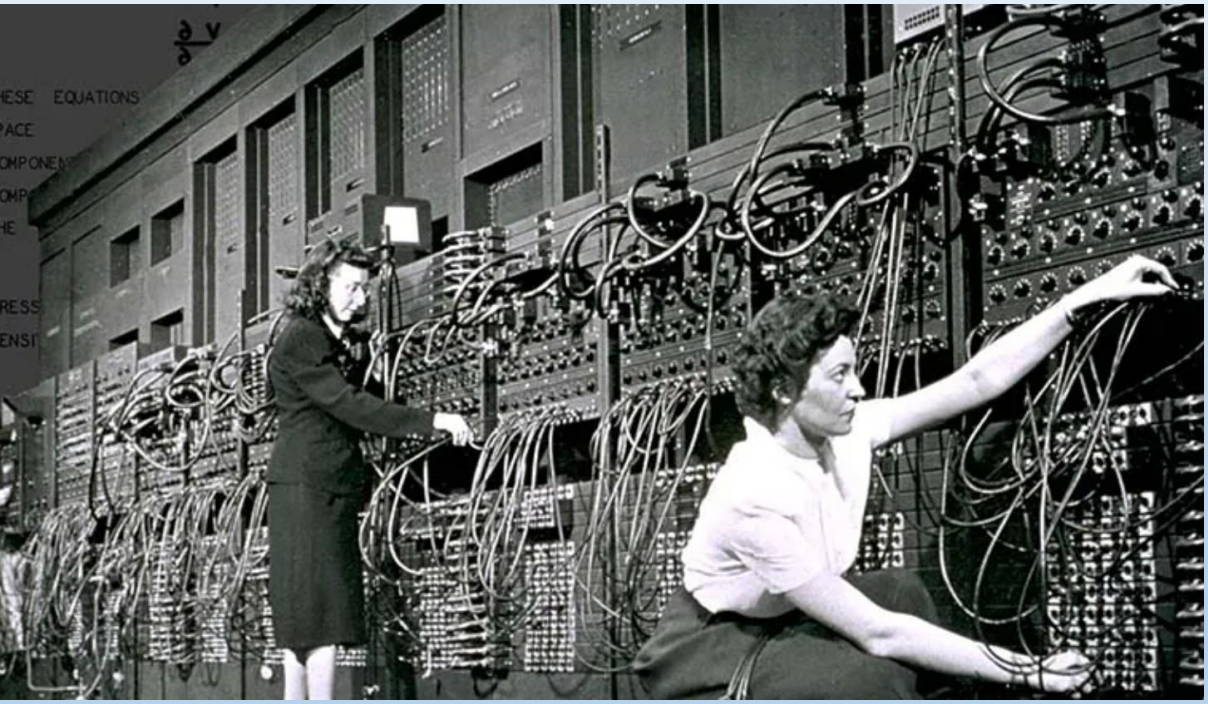
Disciplina: Paradigmas de Linguagens de Programação
Professor: Álvaro Colombero

Objetivos



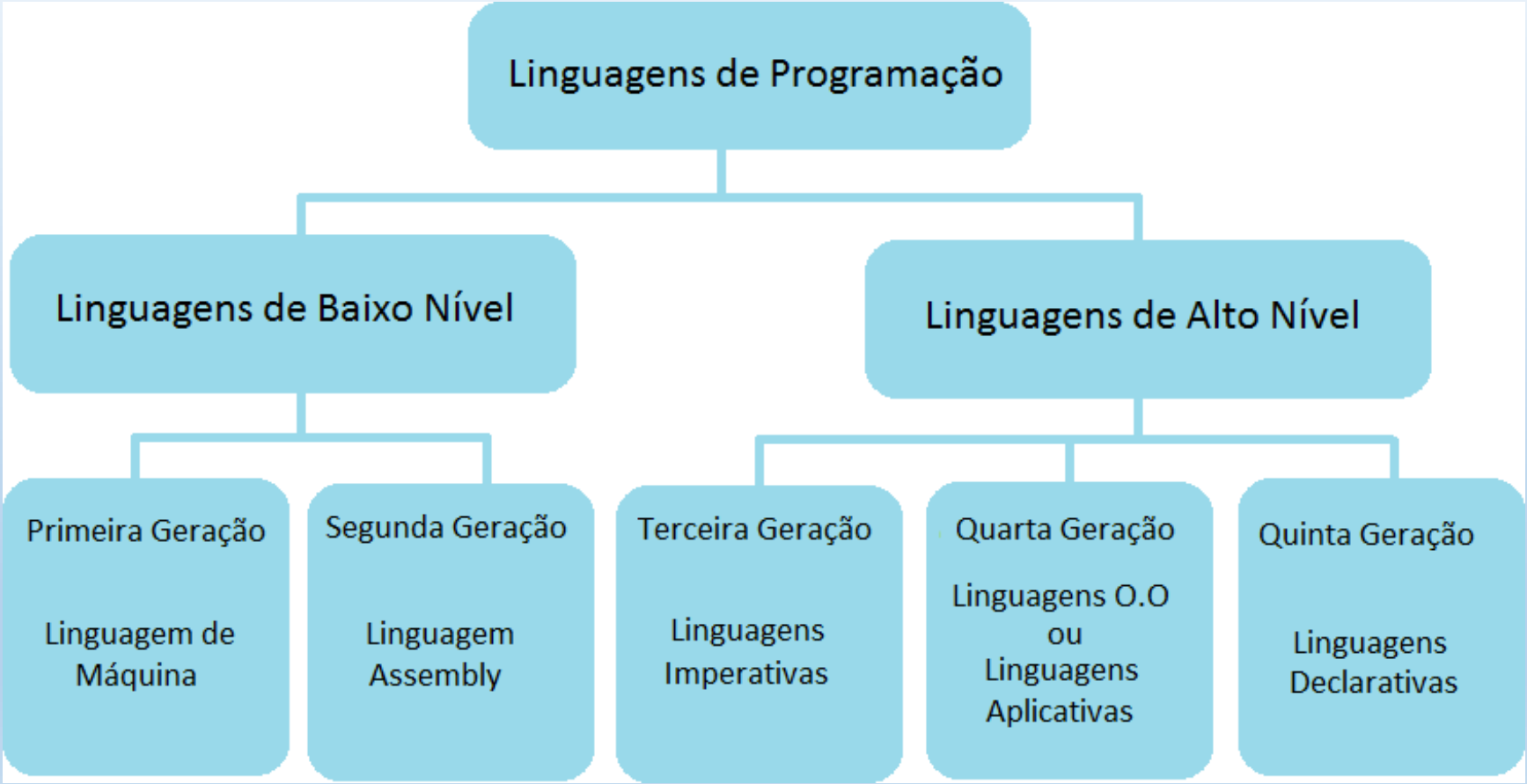
- Compreender os elementos que caracterizam o paradigma imperativo.
- Diferenciar programação estruturada e não estruturada. Conhecer as principais linguagens imperativas.
- Elaborar programas simples usando o comando goto.

Introdução

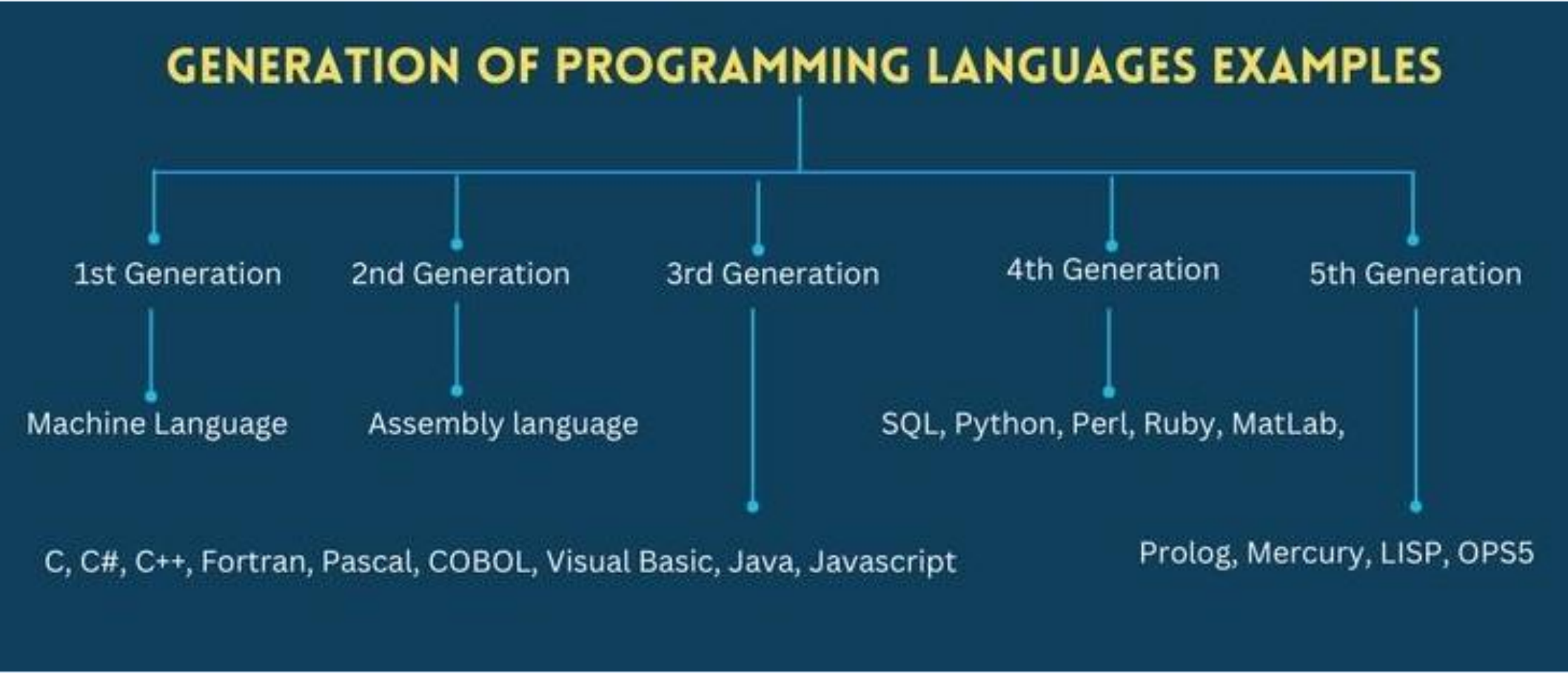


Os primeiros computadores armazenavam os dados na memória e o programa fora dela, geralmente em um painel de fios.

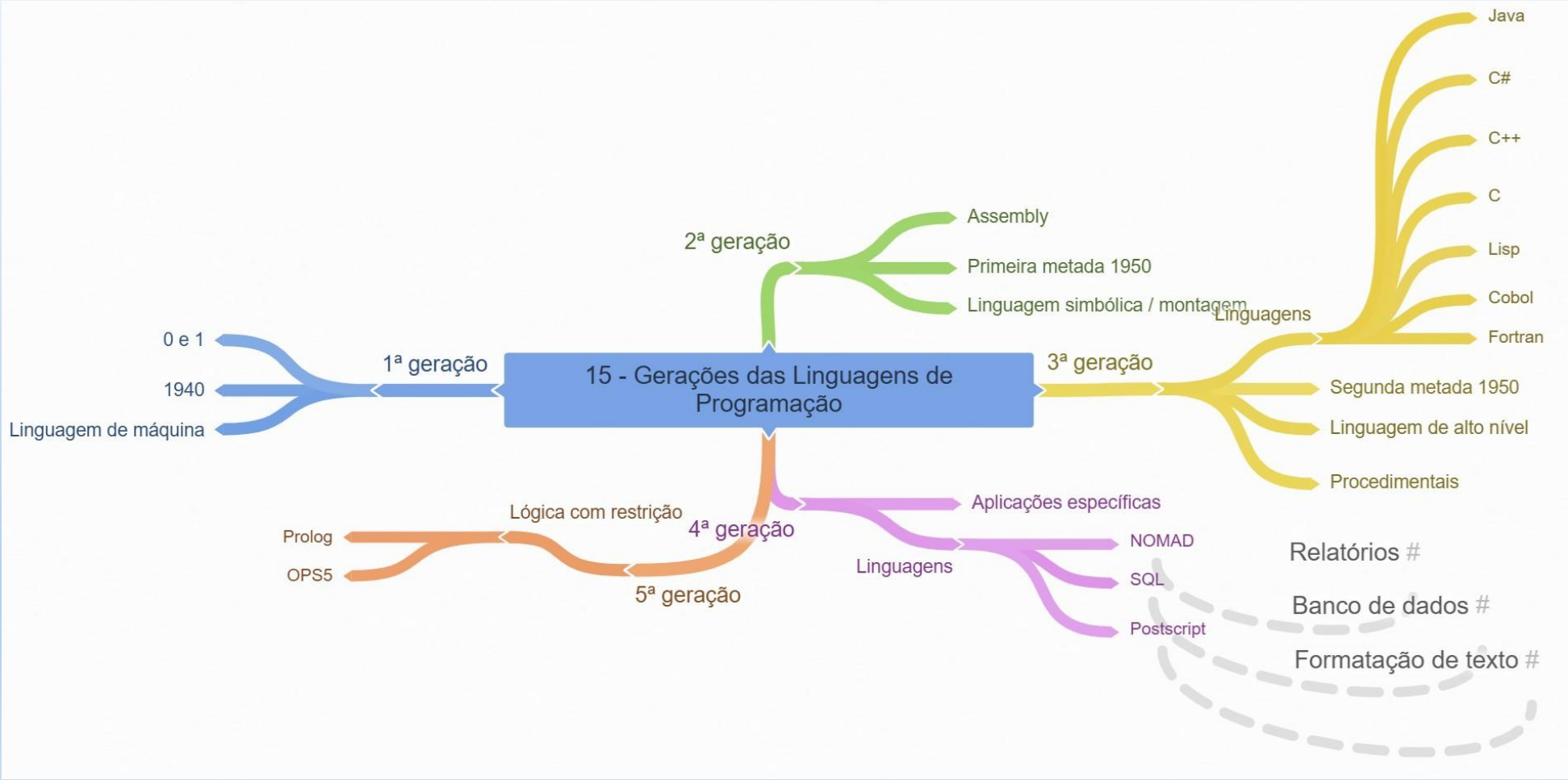
Introdução


Linguagens de Programação
Linguagens de Baixo Nível
Linguagens de Alto Nível
Primeira Geração
Linguagem de Máquina
Segunda Geração
Linguagem Assembly
Terceira Geração
Linguagens Imperativas
Quarta Geração
Linguagens O.O ou Linguagens Aplicativas
Quinta Geração
Linguagens Declarativas

Introdução


GENERATION OF PROGRAMMING LANGUAGES EXAMPLES
1st Generation
2nd Generation
3rd Generation
4th Generation
5th Generation
Machine Language
Assembly language
SQL, Python, Perl, Ruby, MatLab,
C, C#, C++, Fortran, Pascal, COBOL, Visual Basic, Java, Javascript
Prolog, Mercury, LISP, OPS5

Introdução


15 - Gerações das Linguagens de Programação
1ª geração
0 e 1
1940
Linguagem de máquina
2ª geração
Assembly
Primeira metada 1950
Linguagem simbólica / montagem
3ª geração
Linguagens
Java
C#
C++
C
Lisp
Cobol
Fortran
Segunda metada 1950
Linguagem de alto nível
Procedimentais
4ª geração
Lógica com restrição
Prolog
OPS5
5ª geração
Aplicações específicas
Linguagens
NOMAD
SQL
Postscript
Relatórios #
Banco de dados #
Formatação de texto #

Paradigma Imperativo

- Surgiu na década de 40.
- Seus elementos fundamentais estão diretamente relacionados com a arquitetura dos computadores (programas e dados são armazenados na memória e a alteração é feita pelo processador por meio de atribuições).
- Uma instrução é buscada e executada por vez.

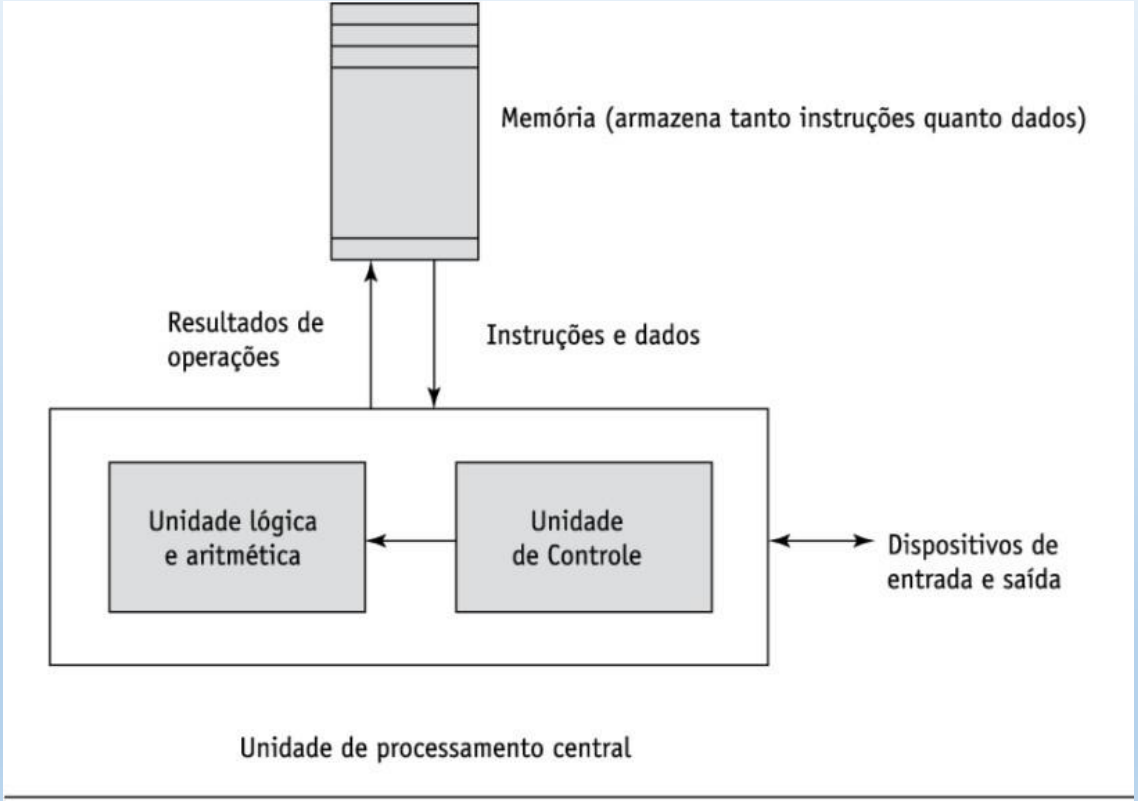


Figura 1.1 A arquitetura de computadores de von Neumann.

Paradigma Imperativo

O que torna uma linguagem imperativa?

Uma linguagem de programação é imperativa se for completa quanto a Turing e possuir as algumas características que surgiram com a evolução das linguagens.



Paradigma Imperativo

- Linguagem completa quanto a Turing:
  - Deve possuir um conjunto mínimo de recursos que pode expressar todos os algoritmos que possam ser concebidos.
- Linguagens imperativas são completas quanto a Turing se possuírem:
  - Variáveis, números inteiros, operações aritméticas, atribuição, sequenciamento de comandos baseado em memória, condições e comandos de ramificação (goto).

Paradigma Imperativo

- Características que surgiram com a evolução das linguagens:
  - Expressões e atribuição.
  - Estruturas de controle (sequencial, seleção e repetição).
  - Entrada/Saída.
  - Manipulação de exceção e erros.
  - Abstração procedural.
  - Suporte de biblioteca para estruturas de dados.

Paradigma Imperativo

- Variáveis associam nomes a posições de memória e identificam o tipo de valor a ser armazenado.
- Expressões recuperam valores armazenados na memória, a partir dos nomes associados e calculam o resultado que também é armazenado em uma posição de memória, por meio de um comando de atribuição.
- O formato padrão de uma atribuição é

  destino = expressão;

- Exemplo:

```
int x = x + 20;
```

Paradigma Imperativo

- Operadores de atribuição mais comuns
  - Estilo Fortran = (C, C++, Java, JavaScript etc).
  - Estilo Algol68 := (Pascal, Ada, Eiffel).
- Expressões são escritas com o uso de operadores aritméticos e lógicos, assim como chamadas a funções fornecidas pelas linguagens.

  - `+  -  *  /  %  div  mod.`
  - `>  <  >=  <=  ==` **OU** `=  !=` **OU** `<>`
  - `and or not` **OU** `&& || !`

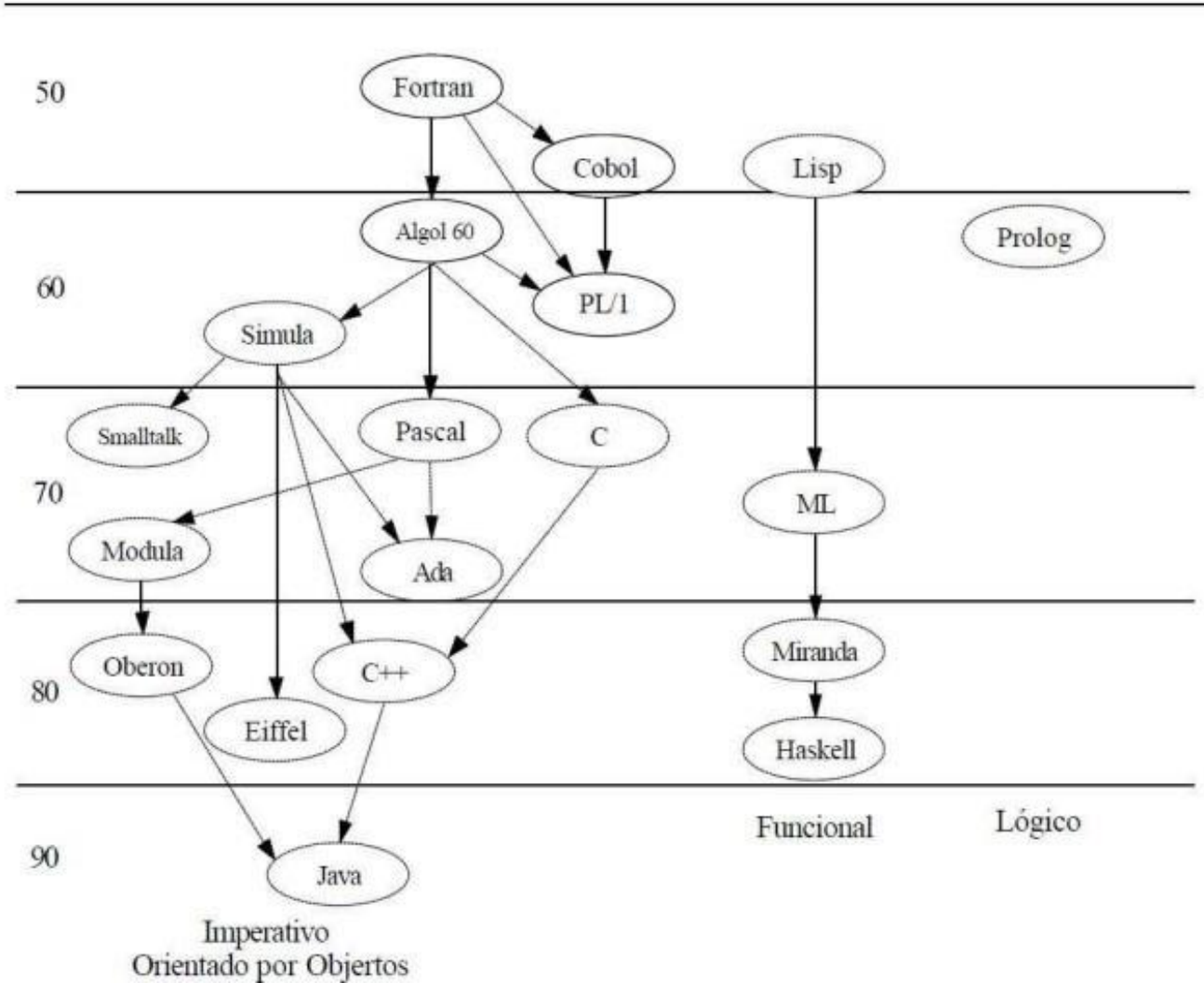
Paradigma Imperativo

- Comandos são executados sequencialmente até um que desvio condicional ou incondicional apareça.
- Inicialmente os comando das linguagens imperativas eram abstrações das instruções de máquina.
- Atualmente eles atuam em um nível de abstração maior.
- O programador eleva ainda mais o nível de abstração, pensando em comandos mais abrangentes (abstração procedural).

Paradigma Imperativo

- Exemplos de Linguagens Imperativas
  - FORTRAN.
  - BASIC.
  - COBOL.
  - ALGOL.
  - Pascal.
  - C.

Classificação das Linguagens: Paradigma



Paradigma Imperativo

- As linguagens do paradigma imperativo podem ser divididas em dois grupos:
  - Estruturado.
  - Não estruturado:
    - Fazem uso de comandos goto ou jump.
    - Servem para determinar a repetição e/ou seleção de instruções.
    - Assembly e Basic.

Paradigma Imperativo

- As linguagens do paradigma imperativo podem ser divididas em dois grupos:
  - Não estruturado.
  - Estruturado.
    - Visa facilitar a leitura e execução de algoritmos.
    - Não fazem o uso do goto (suportam o uso mas oferecem outros mecanismos que dispensam seu uso).
    - Blocos de instruções podem ser selecionados para execução através de declarações de seleção ou repetição.
    - Linguagens estruturadas permitem a criação de subprogramas procedimentos e funções.

Paradigma Imperativo

- Linguagens imperativas são completas quanto a Turing se possuírem:
  - Variáveis, números inteiros, operações aritméticas, atribuição, sequenciamento de comandos baseado em memória, condições e **comandos de ramificação (goto**).
- Dijkstra mostrou que o uso excessivo de **goto** era prejudicial ao desenvolvimento de programas confiáveis e que qualquer sequencia de comando escrita com goto poderia ser escrita com laços condicionais.

Paradigma Imperativo

O que será apresentado pelo programa abaixo?

```
10 PRINT "Iniciando o programa"
20 GOTO 40
30 PRINT "Meio do programa"
40 PRINT "Terminando o programa"
```



Paradigma Imperativo

- Instruções do tipo GOTO (Jump) podem apontar para dois tipos de destinos diferentes:
  - Número de linhas, como no exemplo anterior.
  - Labels, como no próximo exemplo.

```
int main(void) {
  printf("Iniciando o programa\n");
  goto fim;
  printf("Meio do programa\n");
  fim: printf("Terminando o programa\n");
  return 0;   }
```



Atividade 1

Descreva o que faz o programa abaixo e, em seguida, crie uma nova versão dele sem o uso do comando goto.

```
int main() {
    int x = 0;
    repete: printf("%d\n",x);
            x++;
            if (x < 10)
                goto repete;
    return 0;     }
```

Atividade 1

```
int main() {
    int x = 0;
    while (x < 10) {
        printf("%d\n",x);
        x++;
    }
    return 0;
}
```

Bibliografia

- Material complementar:

TUCKER, A. B.; NOONAN, R. E.; **Linguagens de programação: princípios e paradigmas.** 2. ed. São Paulo: McGrall-Hill, 2009. Seções 12.1 a 12.4.

SEBESTA, R. W.; **Conceitos de linguagens de programação**. 9. ed. Porto Alegre: Bookman, 2011. Capítulo 9.



Atividade 2

Usando o comando goto, crie um programa que leia 10 números inteiros, conte e apresente no final quantos são positivos e quantos são negativos. Caso o usuário digite um número 0, o programa deve interromper a execução e finalizar sem mostrar mensagem alguma. Não utilize estruturas de repetição para resolver o problema.